



Centre for Electronic Design and Technology

Netaji Subhas University of Technology , New Delhi

Real-Time Audio Spectrum Analyzer

Submitted by

Ansh Gupta and Shubham Kumar

Date: February 2025

Contents

1	Synopsis	4
2	Introduction	5
3	Apparatus	6
3.1	Hardware	6
3.2	Software	6
4	Block Diagram	7
5	Procedure	8
5.1	Audio Signal Acquisition	8
5.2	ADC Configuration	8
5.3	Sampling Rate Configuration	8
5.4	Sample Collection	9
5.5	Hann Window Application	9
5.6	FFT Computation	10
5.7	Magnitude Calculation	10
5.8	Dominant Frequency Detection	11
5.9	OLED Visualization	11
6	FFT Processing and Analysis	12
6.1	Sampling Frequency	12
6.2	Frequency Resolution	12
6.3	FFT Bin Frequency Mapping	13
6.4	Dominant Frequency Extraction	13
7	Software Implementation	15
7.1	Sampling Section	15
7.2	FFT Processing Section	15
7.3	Display Section	16
8	Results	17
9	Conclusion	18

10 Appendix	20
10.1 Main Software Parameters	20
10.2 Important Frequency Equations	20
10.3 Proposed Final Software Architecture	20

1 Synopsis

This project implements a real-time audio spectrum analyzer using an Arduino microcontroller, an analog microphone sensor, and an OLED display. The system acquires an analog audio signal through the Arduino's Analog-to-Digital Converter (ADC), samples the signal at a rate of 10 kHz, and processes blocks of 128 samples using a Fast Fourier Transform (FFT).

Before performing the FFT, a Hann window is applied to the sampled data to reduce spectral leakage. The FFT converts the audio signal from the time domain into the frequency domain. The magnitude of each frequency bin is calculated, and the bin containing the maximum magnitude is identified as the dominant frequency component.

The detected FFT bin is converted into frequency in hertz and represented as an integer. The display subsystem receives this dominant frequency value and raises the corresponding frequency bar on the OLED display.

The project demonstrates the implementation of an embedded Digital Signal Processing (DSP) system involving real-time data acquisition, timer-based sampling, ADC operation, windowing, FFT computation, frequency extraction, and graphical visualization.

2 Introduction

Sound is a time-varying analog signal containing one or more frequency components. Although the waveform can be observed directly in the time domain, the individual frequencies present in the signal cannot be easily identified from the waveform alone.

A spectrum analyzer converts the signal from the time domain into the frequency domain and displays the distribution of signal energy across different frequencies.

The Fast Fourier Transform (FFT) is an efficient algorithm used to perform this conversion. Instead of analyzing the waveform only as:

Amplitude versus Time

the FFT represents it as:

Magnitude versus Frequency

This project uses an Arduino-based embedded system to implement real-time frequency analysis of an audio signal. An analog microphone sensor provides the input signal to the Arduino ADC. The samples are collected at a fixed sampling rate and stored in a buffer. Once a complete buffer is obtained, a Hann window is applied and a 128-point FFT is performed.

The frequency component with the highest magnitude is then identified. The corresponding FFT bin is converted into an integer frequency value and passed to an independent display function.

The project separates the system into two major functional blocks:

1. Digital Signal Processing Block

- Sampling
- Windowing
- FFT
- Magnitude calculation
- Dominant frequency detection

2. Display Block

- Receive dominant frequency
- Determine corresponding display position
- Raise the corresponding frequency bar

This modular approach separates signal processing from visualization and improves the overall software structure.

3 Apparatus

3.1 Hardware

Table 1: Hardware used in the project

Component	Specification / Purpose
Microcontroller	Arduino Nano
Audio Sensor	KY-037 High Sensitivity Sound Microphone Sensor Module
Display	SSD1306 OLED Display
Connection Hardware	Breadboard and jumper wires
Power Supply	Arduino USB or regulated DC supply

3.2 Software

The following software and libraries are used:

- Arduino IDE
- U8g2 graphics library
- `fix_fft` library
- AVR program memory library
- AVR interrupt library
- SPI library

The program directly accesses the Arduino ADC and Timer1 registers instead of relying entirely on high-level functions such as `analogRead()`.

4 Block Diagram

The overall system architecture of the real-time audio spectrum analyzer is shown in Figure 1.

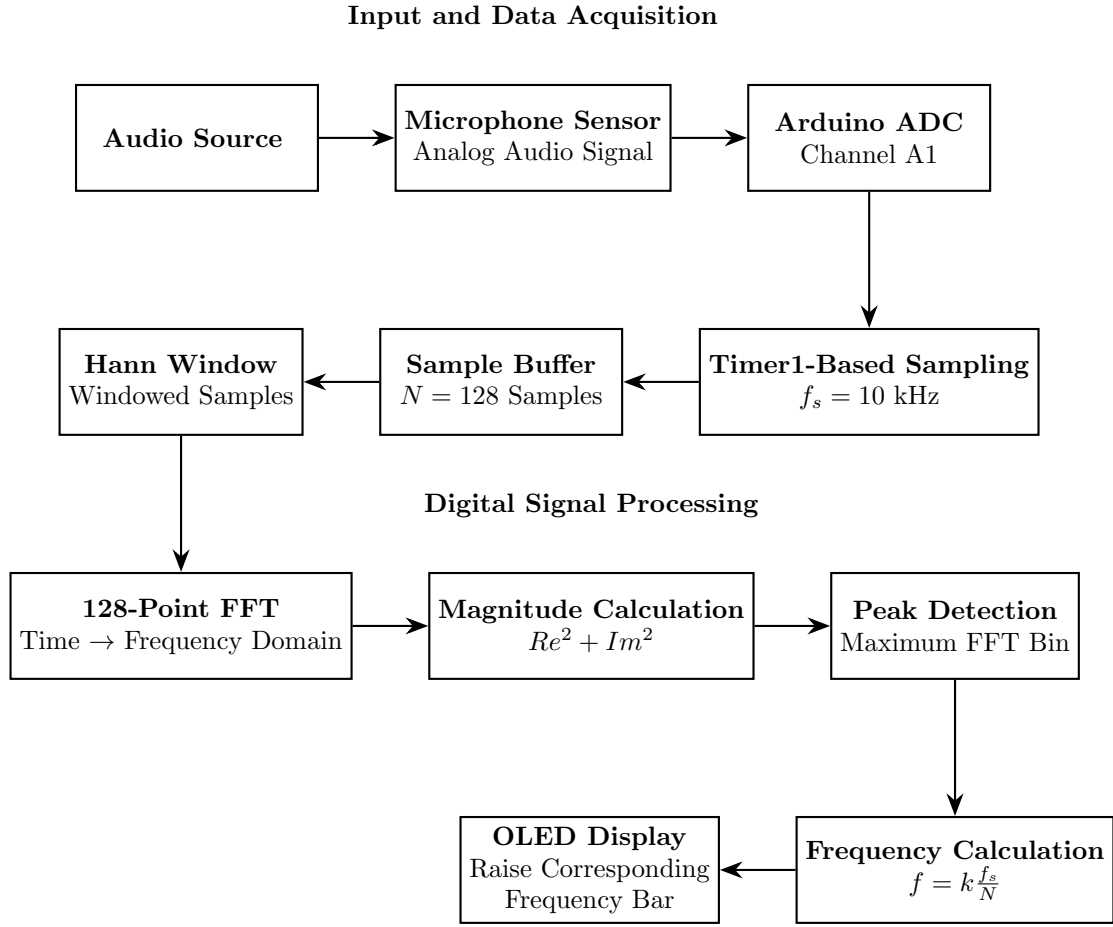


Figure 1: Block diagram of the Arduino-based real-time audio spectrum analyzer

5 Procedure

The following procedure was used to implement the spectrum analyzer.

5.1 Audio Signal Acquisition

An analog microphone sensor was connected to the Arduino analog input channel A1. The microphone converts sound pressure variations into an electrical analog signal.

5.2 ADC Configuration

The Arduino ADC was configured directly through the AVR registers.

The ADC reference voltage was set to AVcc, and ADC channel 1 was selected.

The ADC prescaler was configured to 64.

For a typical Arduino clock frequency of 16 MHz:

$$f_{ADC} = \frac{16 \text{ MHz}}{64}$$

Therefore:

$$f_{ADC} = 250 \text{ kHz}$$

The ADC conversion was initiated directly through the ADC control register.

5.3 Sampling Rate Configuration

Timer1 was configured in CTC mode to generate periodic interrupts corresponding to a sampling rate of:

$$f_s = 10000 \text{ Hz}$$

Therefore, the sampling period is:

$$T_s = \frac{1}{10000}$$

$$T_s = 100 \mu s$$

The Timer1 interrupt schedules a sample request. The actual ADC conversion is performed outside the interrupt service routine.

5.4 Sample Collection

A total of:

$$N = 128$$

samples are collected for every FFT frame.

The 10-bit ADC output ranges from:

$$0 \text{ to } 1023$$

The value is converted into an approximately 8-bit signed value using:

$$\text{Sample} = (\text{ADC} \gg 2) - 128$$

This converts the sampled signal into a format suitable for the `fix_fft` implementation.

5.5 Hann Window Application

Before performing the FFT, a Hann window is applied to the collected audio samples. The purpose of the window is to reduce **spectral leakage** caused by discontinuities between the beginning and end of a finite sample frame.

+++++
Hann window is mathematically defined as:

$$w[n] = 0.5 \left(1 - \cos \left(\frac{2\pi n}{N-1} \right) \right)$$

where:

- n is the sample index, where $0 \leq n \leq N-1$
- N is the total number of samples in one FFT frame
- $w[n]$ is the Hann window coefficient corresponding to sample n

The windowed audio sample is obtained by multiplying each sampled signal value by its corresponding Hann window coefficient:

$$x_w[n] = x[n] \cdot w[n]$$

where:

- $x[n]$ is the original sampled audio signal
- $w[n]$ is the Hann window coefficient
- $x_w[n]$ is the windowed sample supplied to the FFT

For this project, the Hann window coefficients are **precalculated** for all 128 samples instead of being calculated in real time. This avoids repeatedly performing floating-point cosine calculations on the Arduino microcontroller, thereby reducing computational load and improving processing efficiency.

The precalculated Hann window coefficients are stored in the Arduino's program memory (Flash memory) using the `PROGMEM` facility provided by the AVR/Arduino libraries.

The Hann window coefficient array is declared as:

```
1 const uint8_t hannWindow[BUFFER_LEN] PROGMEM = {
2     // Precalculated Hann window coefficients
3 };
```

During processing, the required coefficient is read from Flash memory using:

```
1 uint8_t w = pgm_read_byte(&hannWindow[sampleIndex]);
```

The window is then applied to the sampled audio signal using:

```
1 data[sampleIndex] =
2 ((int16_t)sample * w) / 255;
```

The coefficients are stored as 8-bit integer values scaled approximately between 0 and 255. Therefore, the division by 255 restores the multiplication to the normalized windowing operation.

Using precalculated coefficients stored in Flash memory provides two advantages:

1. It eliminates the need for computationally expensive real-time cosine calculations.
2. It preserves the Arduino's limited SRAM for sample buffers and FFT data.

Therefore, the Hann window processing stage can be represented as:

$$\boxed{x[n]} \xrightarrow{\text{Multiply by } w[n]} \boxed{x_w[n]} \rightarrow \boxed{\text{FFT}}$$

5.6 FFT Computation

After collecting all 128 samples, a 128-point FFT is performed.

```
1 fix_fft(data, im, 7, 0);
```

Since:

$$2^7 = 128$$

the parameter 7 specifies a 128-point FFT.

5.7 Magnitude Calculation

After the FFT, every frequency bin contains a real and imaginary component.

The magnitude squared is calculated as:

$$\text{Magnitude}^2 = Re^2 + Im^2$$

The square root is not required for peak detection because comparing magnitude squared produces the same peak location as comparing the actual magnitude.

5.8 Dominant Frequency Detection

The frequency bin having the highest magnitude is identified.

If the maximum magnitude occurs at FFT bin k , the dominant frequency is calculated as:

$$f = k \times \frac{f_s}{N}$$

For this project:

$$f = k \times \frac{10000}{128}$$

Therefore:

$$f = k \times 78.125 \text{ Hz}$$

The resulting frequency is stored as an integer and passed to the display function.

5.9 OLED Visualization

The display function receives the dominant frequency as input.

Conceptually:

```
1 int peakFrequency = calculatePeakFrequency();
2
3 displayPeakFrequency(peakFrequency);
```

The display function converts the received frequency into the corresponding display position and raises the associated frequency bar.

6 FFT Processing and Analysis

6.1 Sampling Frequency

The sampling frequency used in the project is:

$$f_s = 10 \text{ kHz}$$

According to the Nyquist criterion, the maximum frequency that can theoretically be represented is:

$$f_{\max} = \frac{f_s}{2}$$

Therefore:

$$f_{\max} = 5 \text{ kHz}$$

The analyzer is therefore designed to process frequencies approximately within the range:

$$0 \text{ Hz to } 5000 \text{ Hz}$$

6.2 Frequency Resolution

The frequency resolution of an FFT is given by:

$$\Delta f = \frac{f_s}{N}$$

For:

$$f_s = 10000 \text{ Hz}$$

and:

$$N = 128$$

the resolution is:

$$\Delta f = \frac{10000}{128}$$

$$\boxed{\Delta f = 78.125 \text{ Hz}}$$

Therefore, adjacent FFT bins are separated by approximately 78.125 Hz.

6.3 FFT Bin Frequency Mapping

The frequency corresponding to an FFT bin is:

$$f_k = k \cdot \frac{f_s}{N}$$

Some example values are shown below.

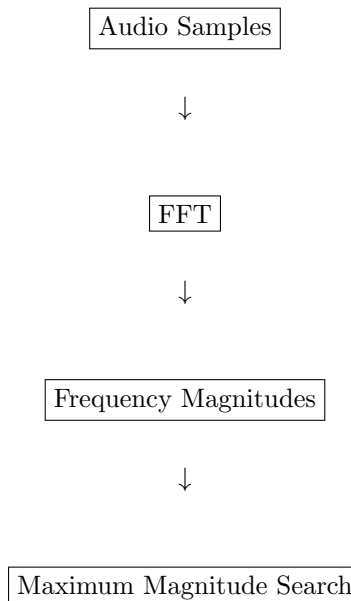
Table 2: FFT bin to frequency relationship

FFT Bin	Approximate Frequency
0	0 Hz
1	78 Hz
2	156 Hz
5	390 Hz
10	781 Hz
20	1562 Hz
30	2343 Hz
40	3125 Hz
50	3906 Hz
60	4687 Hz
64	5000 Hz

Only the first half of the FFT output contains unique frequency information because the input audio signal is real-valued.

6.4 Dominant Frequency Extraction

The processing flow is:



↓

Peak FFT Bin

↓

Bin-to-Frequency Conversion

↓

Integer Frequency

Instead of the display system requiring access to FFT data, real values, imaginary values, and magnitude calculations, it only receives:

```
1 int frequency;
```

This provides a clean separation between the signal processing and visualization sections.

7 Software Implementation

The software is divided into three major sections.

7.1 Sampling Section

This section is responsible for:

- Timer configuration
- Sample scheduling
- ADC conversion
- Sample buffer storage

The conceptual operation is:

Timer1 → Sample Request → ADC Conversion → Store Sample

7.2 FFT Processing Section

This section is responsible for:

- Performing the FFT
- Calculating frequency magnitudes
- Detecting the peak FFT bin
- Converting the bin into frequency

The conceptual function is:

```
1 int calculatePeakFrequency();
```

The output of this function is the dominant frequency in hertz.

The function hides the lower-level implementation details such as:

- FFT arrays
- Real components
- Imaginary components
- Magnitude calculations
- Peak searching

7.3 Display Section

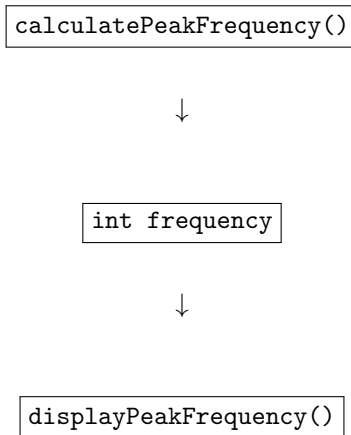
The display section receives only the extracted frequency:

```
1 void displayPeakFrequency(int frequency);
```

The display subsystem is responsible for:

- Frequency-to-bin conversion
- Bin-to-pixel conversion
- OLED graphics
- Raising the corresponding frequency bar

The final architecture becomes:



This creates a clean separation between digital signal processing and visualization.

8 Results

The implemented system successfully performs real-time sampling and frequency-domain analysis of an incoming audio signal.

The main characteristics of the system are shown below.

Table 3: Spectrum analyzer specifications

Parameter	Value
Sampling Rate	10 kHz
FFT Length	128 Samples
Frequency Resolution	78.125 Hz
Maximum Theoretical Frequency	5 kHz
ADC Resolution	10-bit Input
FFT Input Resolution	Signed 8-bit
ADC Channel	A1
Display	128 × 64 OLED
Window Function	Hann Window

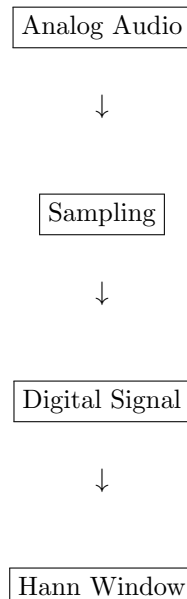
The system converts the input audio signal into frequency-domain information and identifies the frequency bin with the maximum magnitude.

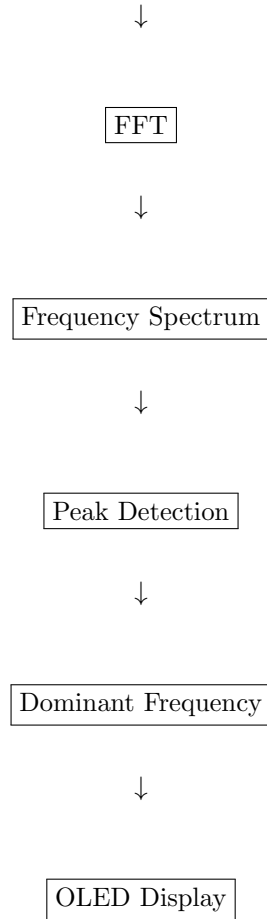
The detected bin is converted into an integer frequency according to:

$$f_{\text{peak}} = k_{\text{peak}} \times \frac{10000}{128}$$

The resulting frequency is supplied to the OLED display subsystem, where the corresponding frequency position is highlighted.

The complete signal processing chain is:





9 Conclusion

The project successfully demonstrates the implementation of a real-time audio frequency analyzer using an Arduino microcontroller and Fast Fourier Transform processing.

An analog audio signal is acquired through the Arduino ADC and sampled at 10 kHz using Timer1-based sample scheduling. A buffer of 128 samples is collected and processed using a Hann window followed by a 128-point FFT.

The FFT converts the sampled audio signal from the time domain into the frequency domain. The magnitude of the frequency bins is calculated, and the bin containing the maximum magnitude is identified. The corresponding bin index is converted into an integer frequency value.

The software architecture separates the system into two independent functional layers: frequency calculation and display. The FFT processing function extracts the dominant frequency, while the display function uses this frequency to raise the corresponding bar on the OLED.

The final system demonstrates practical embedded implementation of:

- Analog-to-Digital Conversion
- Timer-based sampling
- Digital Signal Processing

- Hann windowing
- Fast Fourier Transform
- Frequency-domain analysis
- Peak frequency detection
- OLED visualization
- Modular software design

The project provides a practical foundation for more advanced systems such as audio analyzers, musical note detectors, pitch detection systems, acoustic monitoring devices, and embedded DSP applications.

10 Appendix

10.1 Main Software Parameters

```
1 #define SAMPLE_RATE 10000UL
2 #define BUFFER_LEN 128
3 #define AVG_FRAMES 4
4 #define GROUP_SIZE 2
```

10.2 Important Frequency Equations

Nyquist Frequency

$$f_{\text{Nyquist}} = \frac{f_s}{2}$$

For this project:

$$f_{\text{Nyquist}} = 5000 \text{ Hz}$$

FFT Frequency Resolution

$$\Delta f = \frac{f_s}{N}$$

For this project:

$$\Delta f = 78.125 \text{ Hz}$$

Frequency of FFT Bin

$$f_k = k \frac{f_s}{N}$$

Magnitude Squared

$$\text{Magnitude}^2 = Re^2 + Im^2$$

10.3 Proposed Final Software Architecture

```
1 void loop()
2 {
3     // Collect samples
4
5     if (fftReady)
6     {
7         int dominantFrequency =
8             calculatePeakFrequency();
```

```
9
10     displayPeakFrequency(
11         dominantFrequency
12     );
13 }
14 }
```